

In the first session, participants typed responses independently without any aids, showcasing their intrinsic typing abilities and cognitive processes. The second session saw the same participants using internet access and AI tools like ChatGPT to mimic conditions of potential academic misconduct. The questions posed were designed to elicit different levels of cognitive load, covering both general (opinion-based) and science-specific (fact-based) subjects that required analytical and methodological reasoning. Responses that did not meet established quality standards, such as a minimum word count, were omitted to preserve the integrity of the dataset.

Prior research has demonstrated that users exhibit distinct writing patterns when engaging with familiar versus unfamiliar topics [47]. Specifically, writers take longer pauses to reflect on their text and organize their thoughts when confronted with less familiar topics, often using the backspace key to enhance textual coherence. In contrast, writing on familiar topics generally proceeds more rapidly and with greater consistency [48]. Accordingly, our study was designed to include two targeted sections, *Opinion-based* and the *Fact-based*, to explore these variations in writing behavior.

The *Opinion-based* section comprised two open-ended essay questions addressing broad topics in society, environment, and current affairs. Participants were tasked with arguing for or against a specific proposition, such as, *'Is AI a promising tool that can revolutionize industries and improve human life, or does it carry significant ethical risks, including job displacement and loss of control?'*

Meanwhile, the *Fact-based* section included two scenario-driven questions centered on high-school science concepts. Participants were required to elaborate on their observations and rationalize the underlying scientific principles. For instance, one question posed was, *'You are given a mixture of sand and salt. Design an experiment to separate the two components using appropriate laboratory equipment and techniques. Detail your step-by-step procedure and the principles underlying the separation process.'*

Participants crafted responses to four questions each session, structured to elicit one of six cognitive load levels defined by Balagani et al. [29]. Each typing session, capturing detailed keystroke dynamics such as key-down and key-up events, spanned 30 to 45 minutes. Participants were required to compose answers with a minimum of 300 characters per question and were permitted to refine their responses using the Delete or Backspace keys. This data, recorded by high-precision keystroke sensors accurate to ± 200 microseconds, constitutes the Proposed dataset further detailed in our study.

Employing three distinct datasets facilitated a comprehensive analysis across diverse typing scenarios—free, fixed, bona fide, and assisted—thereby enabling a nuanced understanding of keystroke dynamics within academic settings. The targeted methodology of the custom dataset creation, in conjunction with the comparative analysis against the SBU and Buffalo datasets, enhances this understanding and contributes to the integrity of educational assessments.

3.2. Training detection models

To analyze keystroke dynamics, we implemented a deep learning-based approach utilizing an LSTM architecture, specifically the widely recognized LSTM model, TypeNet.

TypeNet [49] is a Long Short-Term Memory network developed to authenticate users by analyzing their typing patterns, including metrics such as typing speed, rhythm, and key press duration. The LSTM component is essential in TypeNet, as it retains the memory of previous typing activities, significantly enhancing its ability to discern and predict an individual's distinctive typing style, particularly in extensive texts. Engineered to be compatible with various input devices, both physical and touchscreen, TypeNet is adaptable across different platforms. It was rigorously evaluated using three distinct loss functions: softmax, contrastive, and triplet loss, which contributes to its versatility in application. For detailed architectural information, please refer to [49]. Leveraging the capabilities of TypeNet, we adapted this model for our plagiarism detection initiatives, utilizing keystroke dynamics. Subsequent paragraphs will elaborate on the modifications made to the TypeNet architecture for this purpose.

System architecture: We enhanced the TypeNet architecture by integrating a Siamese network, which processes sequence pairs to differentiate between bona fide and assisted writing. The modifications to the architecture include: (1) Two LSTM layers, each producing a 128-dimensional output, complemented by batch normalization layers and tanh activation functions. (2) A fully connected layer succeeding each LSTM, designed to map the embeddings to a uniform 128-dimensional space. (3) Dropout layers added post each LSTM layer to mitigate overfitting, with a dropout rate of 0.5 and a recurrent dropout of 0.2. (4) The sequence length M and batch size were varied during experiments, with M ranging from 25 to 500 and batch sizes from 32 to 512. We explored different training durations, observing that performance typically stabilized between 50 and 100 epochs. The Adam optimizer was deployed with an adaptable learning rate ranging from 0.0001 to 0.005, and L_2 regularization was applied to prevent over-fitting further.

Data Preprocessing: We standardized the sequence lengths to a predetermined length M by padding shorter sequences and clipping longer ones while adjusting M during hyperparameter tuning. The value of M ranged between 25 and 500, with batch sizes varying between 32 and 512, in powers of 2. Sequences with excessive use of the Shift key (over 20%) and those significantly shorter than M (less than 50%) were excluded. Each keystroke sequence was characterized by three attributes: timestamps, keycodes, and key actions (KeyDown, KeyUp), normalized as follows: Timestamps were scaled to a range of $[0, 1]$ using min-max normalization. Keycodes were normalized by dividing each by 255. Key actions were binary encoded, with KeyDown as 0 and KeyUp as 1. Siamese networks were trained using pairs of sequences from fixed and free-text contexts. Opposite sequence pairs were labeled 1, and similar sequence pairs 0. We maintained a balanced dataset to minimize biases.

Loss Function: Contrary to the original TypeNet, which utilized Euclidean distance for comparisons, our model incorporates a modified loss function that employs cosine

similarity to assess the closeness of embeddings from paired sequences. Training leverages Binary Cross Entropy Loss, with adjustments reflecting the cosine similarities. The decision threshold for classifying sequences as bona fide or assisted is determined by the Equal Error Rate (EER) point, derived from ROC analysis, which optimizes the balance between false acceptance and rejection rates. This threshold is then used to calculate the predicted output’s Accuracy and F_1 score.

3.3. Evaluation scenarios

We assessed TypeNet’s effectiveness as a plagiarism detector through various evaluation setups, focusing on the user, keyboard, context, and datasets. Each scenario utilized specific and agnostic modeling to measure the model’s adaptability and accuracy under diverse conditions.

We explored both *user-specific* and *user-agnostic* scenarios. In the *user-specific* evaluation, we trained and tested the model independently with an 80-20 split for each user’s data, ensuring no overlap between the training and testing datasets. We aggregated all possible user sequences from the datasets, then divided each sequence for training and testing to assess the model’s performance on sequences from users previously trained with different data from the same user. In the *user-agnostic* approach, we assessed the model’s performance across various datasets to gauge its consistency and generalizability. This involved creating a divided environment by splitting the entire set of users into distinct training and testing groups, ensuring no user data overlapped between groups. We varied the ratios for training, validation, and testing, such as 50-25-25 and 80-10-10, to evaluate the model’s efficacy across diverse user groups.

In the *keyboard-specific* setup, the model was trained and tested on data collected from the same keyboard type with an 80-20 split, ensuring distinct sequences in each set. The *keyboard-agnostic* evaluation involved training the model using data from three keyboard types and testing it on a fourth. This approach evaluated the model’s ability to generalize across different keyboard types, capturing the subtleties of keystroke dynamics regardless of the keyboard used. For this evaluation, we employed the Buffalo dataset, which categorizes data based on keyboard type, differentiating between free and fixed sequences for detection purposes. For convenience, the *Lenovo* keyboard is labeled as K_0 , *HP wireless* as K_1 , *Microsoft* as K_2 , and the *Apple Bluetooth* as K_3 .

Context-specific modeling entailed training and testing the model on homogeneous datasets, which maintained content consistency within each set. This approach assessed the model’s effectiveness in managing data with similar linguistic contexts. In context-specific scenarios, we trained and tested the model on sequences from the same context, ensuring there was no overlap of sequences between the training and testing sets. An 80-20 split was implemented for sequences from each context for training and testing purposes. Conversely, *context-agnostic* modeling presented a challenge to the model by training on datasets that covered two different topics and testing on a third, previously unseen topic. This method evaluated the model’s ability to adapt and perform linguistic analysis across diverse contexts, assessing its proficiency with familiar and new content. For this evaluation, we utilized the SBU dataset, which is divided into three parts based

on context: Gay Marriage, Gun Control, and Restaurant Feedback, labeled as *GM*, *GC*, and *RF*, respectively.

In the *dataset-specific* approach, we train and test the model using the same dataset, ensuring no sequence overlap between the training and testing sets. This method evaluates the model’s performance within a consistent typing environment. Conversely, the *dataset-agnostic* approach involves training the model on a combination of datasets and testing it on different datasets or combinations thereof. This strategy assesses the model’s ability to generalize and maintain effectiveness across diverse environments, which may differ significantly in their curation. By employing both evaluation methods, we aim to comprehensively understand the model’s strengths and weaknesses across various data collection environments, ensuring its suitability and generalizability for real-world applications.

3.4. Evaluation metrics

We measured and reported Accuracy, F_1 score, False Acceptance Rate (FAR), and False Rejection Rate (FRR) for each detector and scenario.

Accuracy assesses the effectiveness of the plagiarism detection system by calculating the ratio of correctly identified documents (both true positives and true negatives) to the total number of evaluated documents. High Accuracy indicates that the system effectively distinguishes between assisted (true positives) and bona fide (true negatives) submissions, ensuring fair student evaluations.

F_1 score represents the harmonic mean of precision and recall, providing a balanced measure of the system’s precision and ability to identify all actual instances of plagiarism. A strong F_1 score implies that the system reliably identifies a high percentage of true plagiarism cases while maintaining a low rate of false positives. This balance is crucial to prevent wrongful accusations against students, which could damage their academic reputation and trust in the educational system.

False Acceptance Rate (FAR) reflects the likelihood that the system will fail to detect actual plagiarism, incorrectly labeling a plagiarized document as bona fide. It is essential to reduce FAR to ensure that all students are assessed fairly and that genuine efforts are acknowledged properly.

False Rejection Rate (FRR) measures the probability that the system will mistakenly classify a bona fide document as plagiarized. Maintaining a low FRR is vital to protect students from unjust accusations.

4. Results and discussion

We present the performance of each model across all evaluation scenarios in Tables 2 and 3, which are further explained in the subsequent paragraphs.

The *user-specific* models, where the training and testing sets include keystroke sequences from the same user with no overlap, achieved Accuracy and F_1 scores of 81.86% and 81.85%. FAR and FRR were recorded at 23.71% and 26.24%, respectively, as illustrated in Table 2.

Conversely, *user-agnostic* models achieved an accuracy ranging from 63.56% to 66.54%. The FAR and FRR ranged from 38.82% to 42.51% and 39.57% to 39.86%, respectively (Table 3).

Table 2. The table summarizes the performance of the TypeNet architecture across various datasets and scenarios, highlighting metrics such as Accuracy, F_1 scores, FAR, and FRR. It encompasses keyboard-specific scenarios (K_0, K_1, K_2, K_3 represent different keyboards), context-specific scenarios (GM, GC, RF), user-specific merged data from all datasets, and dataset-specific evaluations (SBU - S, Proposed - P, Buffalo - B). Notably, the lowest observed Accuracy is 74.98% in the keyboard-specific K_3 scenario, whereas the highest reaches 85.72% in the dataset-specific Buffalo scenario. F_1 scores span from a low of 73.34% (K_3) to a high of 84.72% (B). The merged user-specific data demonstrates robust performance, with both Accuracy and F_1 score exceeding 81%. Among dataset-specific evaluations, the Buffalo dataset stands out as the top performer, achieving an Accuracy of 85.72% and an F_1 score of 84.72%.

Dataset		TypeNet			
Train	Test	Acc.	F_1	FAR	FRR
<i>Keyboard Specific</i>					
K_0	K_0	84.64	83.45	25.38	19.83
K_1	K_1	80.02	78.91	29.11	24.19
K_2	K_2	77.77	76.58	32.14	25.12
K_3	K_3	74.98	73.34	32.6	30.16
<i>Context Specific</i>					
GM	GM	80.24	81.52	30.81	21.27
GC	GC	79.39	80.67	34.62	22.33
RF	RF	76.52	78.01	34.02	31.13
<i>User Specific</i>					
Merged	Merged	81.86	81.85	23.71	26.24
<i>Dataset Specific</i>					
S	S	80.85	83.31	18.83	32.93
P	P	81.04	82.16	22.6	28.2
B	B	85.72	84.72	17.64	23.91

These findings indicate that the model performs better with keystroke sequences from the same user than those from different users, demonstrating limited generalizability across user keystroke patterns. This is consistent with the significant variation in typing behavior among users, even when typing identical content—offering insights into the effectiveness of continuous authentication through keystroke dynamics.

In *keyboard-specific* scenarios, where the keyboard type is consistent and known, the models achieve an accuracy in the range of 74.98% to 84.64%, and F_1 scores range from 73.34% to 83.45%.

In *keyboard-agnostic* scenarios, the models attained an accuracy between 78.11% and 80.54%, and F_1 scores between 76.89% and 78.77%. In both cases, the FAR and FRR remain below 30%. The minimal differences in accuracy, F_1 scores, and error rates suggest that the proposed plagiarism detector is robust regardless of the type of keyboard used.

In *context-specific* scenarios (see Table 2), TypeNet achieved an Accuracy in the range of 76.52% to 80.24%, F_1 scores from 78.01% to 81.52%, FAR from 30.81% to 34.62%, and FRR from 21.27% to 31.13%. These metrics illustrate

the model’s effectiveness in environments that align with its training conditions. However, performance slightly drops in *context-agnostic* scenarios (see Table 3). Accuracy ranged from 70.21% to 78.67%. F_1 scores were between 70.23% and 78.24%. FAR increased, ranging from 32.30% to 39.65%, and FRR extended from 24.10% to 34.73%. The performance remains strong despite these reductions, indicating only modest declines from the context-specific results. This underscores TypeNet’s adaptability across varying contexts.

Table 3. The performance of the TypeNet architecture across various platform-agnostic scenarios is summarized in the table, with metrics including Accuracy, F_1 scores, FAR, and FRR. The Merged data consists of user sequences combined from all datasets, where "S" denotes the SBU dataset, "P" stands for the Proposed dataset, and "B" for the Buffalo dataset. The architecture achieves its best and most consistent performance in keyboard-agnostic scenarios, with Accuracy ranging from 78.11% to 80.54% and robust F_1 scores. Context-agnostic scenarios exhibit variability, with the lowest Accuracy recorded at 70.21%. In user-agnostic scenarios, performance notably drops below 70% in certain configurations, reflecting the model’s challenges in adapting to diverse user behaviors. Dataset-agnostic testing uncovers the architecture’s difficulties in generalizing across unfamiliar data, leading to significant performance reductions, especially in configurations where training and testing datasets do not overlap, yielding the lowest accuracies and highest error rates.

Dataset		TypeNet			
Train	Test	Acc.	F_1	FAR	FRR
<i>Keyboard Agnostic</i>					
$K_{0,1,2}$	K_3	78.11	75.88	28.04	29.01
$K_{0,1,3}$	K_2	79.71	78.06	27.5	28.37
$K_{0,2,3}$	K_1	80.54	78.77	24.01	28.37
$K_{1,2,3}$	K_0	78.96	76.89	29.15	24.95
<i>Context Agnostic</i>					
(GM, RF)	GC	72.96	72.4	38.3	28.56
(GC, RF)	GM	78.67	78.24	32.3	24.1
(GM, GC)	RF	70.21	70.23	39.65	34.73
<i>User Agnostic</i>					
Merged (80-10-10)		63.56	62.98	42.51	39.57
Merged (50-25-25)		66.54	66.22	38.82	39.86
<i>Dataset Agnostic</i>					
(S, P)	B	68.72	66.21	33.13	39.66
(S, B)	P	73.23	72.65	27.95	40.22
(P, B)	S	52.24	61.86	47.53	48.51
S	(P, B)	59.73	57.03	41.36	44.29
P	(S, B)	56.17	54.13	44.95	45.72
B	(S, P)	53.57	53.16	46.32	48.01

In *dataset-specific* scenarios, the TypeNet model demonstrates significant improvement, with each case achieving an accuracy and F_1 score above 80%, and the highest accuracy recorded at 85.72%. FAR and FRR were observed below 23% and 30%, with the lowest values at 17.64% and 23.91%, respectively (see Table 3). This indicates strong performance when the model is trained